

Google SecOps Operation Handbook

Log Ingestion Detection Engineering SOAR Reference

Log Ingestion Detection Engineering SOAR Reference.....	1
1. Introduction.....	3
1.1 Purpose of this Handbook.....	3
1.2 Scope.....	3
1.3 Document Conventions.....	3
2. Platform Overview.....	4
2.1 What Google SecOps Is.....	4
2.2 Core Components.....	4
2.3 High-Level Data Flow.....	4
3. Log Ingestion Architecture.....	6
3.1 Ingestion Method.....	6
3.2 Log Source Inventory.....	6
4. Detection Engineering.....	8
4.1 Detection Types.....	8
4.2 Active Rule Catalog — This Project.....	8
4.3 Curated Detection Library (Reference Only).....	10
5. SOAR Playbooks.....	11
5.1 Case Tagging.....	11
5.2 Okta User Enrichment.....	12
5.3 Github Case Assignment.....	12
5.4 Design Observations.....	13
6. Case Management & Triage Workflow.....	15
6.1 Case Lifecycle.....	15
6.2 Tagging Taxonomy.....	15
6.3 SLA Tiers.....	16
7. Best Practices:.....	17
7.1 Log Ingestion.....	17
7.2 Detection Engineering (YARA-L Rules).....	17
7.3 SOAR Playbooks.....	18
7.4 Case Management & Triage.....	18
7.5 General Platform Hygiene.....	19
8. Appendices.....	20
Appendix A — Curated Detection Rules Catalog.....	20
Appendix B — Glossary.....	20

1. Introduction



1.1 Purpose of this Handbook

This handbook documents the current Google SecOps (Chronicle) deployment for **LabelBox**, the log sources ingested, the detection rules actively producing cases, and the SOAR playbooks that automate triage. It is intended as a living reference for security engineers operating, extending, or handing off this environment.

1.2 Scope

This document covers four areas of the platform:

- **Log ingestion** — which sources are connected and how.
- **Detection engineering** — the rules currently generating cases in this environment.
- **SOAR automation** — the playbooks that enrich, tag, assign, and route cases.
- **Case management** — the stages, tagging conventions, and SLA tiers those playbooks enforce.

The full curated detection-rule library shipped with Google SecOps (covering many additional integrations such as AWS, Azure, CrowdStrike, O365, and others) is tracked separately in an accompanying spreadsheet and is referenced but not reproduced here — see Appendix A.

1.3 Document Conventions

- **UDM** — Unified Data Model, Chronicle's normalized event schema. All ingested logs are parsed into UDM before rules or playbooks can act on them.
- **YARA-L** — The rule language used to write custom Chronicle detections.
- **Case** — A SOAR-level grouping of one or more related alerts.
- **Alert** — A single detection-rule match on one or more UDM events.

2. Platform Overview

2.1 What Google SecOps Is

Google SecOps is Google’s cloud-native SIEM and SOAR platform (built on the former Chronicle and Simplify products). It combines large-scale log storage and normalization, a detection engine that runs both curated and custom rules against that normalized data, and an integrated SOAR layer that turns rule matches into cases and can execute automated response playbooks against them.

2.2 Core Components

Ingestion & UDM: Raw logs from each connected source are parsed into the Unified Data Model, giving every source a common event schema regardless of vendor.

Detection Engine: YARA-L rules — both Google-curated and custom-authored — run continuously against UDM events and raise alerts on a match.

Case Management: Related alerts are grouped into cases, each with a priority, a numeric risk score, a stage (e.g. Investigation), and an assignee.

SOAR / Playbooks: Automated workflows (documented in Section 5) that react to case or alert creation and perform enrichment, tagging, assignment, SLA-setting, and (per the current playbooks) Gemini-assisted case summarization.

2.3 High-Level Data Flow

Step	What happens
1. Collection	Each source (Section 3) pushes or is polled for logs using Google’s standard, documented feed method for that product — no third-party log shipper is used in this deployment.
2. Parsing	Chronicle parses raw logs into UDM using the built-in parser for that log type (e.g. GCP_CLOUDAUDIT, OKTA, GITHUB).

Step	What happens
3. Detection	Active YARA-L rules (Section 4) evaluate UDM events and raise alerts on matches.
4. Case creation	Alerts are grouped into a case with a priority and risk score.
5. SOAR Automation	Playbooks matching the case/alert trigger conditions execute automatically (Section 5).
6. Analyst triage	The case reaches an analyst pre-tagged, pre-assigned, and (where applicable) pre-scoped with an SLA.

3. Log Ingestion Architecture

3.1 Ingestion Method

All log sources in this deployment are ingested using Google SecOps’ standard, natively-documented ingestion methods for each product (API-based feeds configured directly in the platform). This environment does not use a third-party log-shipping/forwarding layer (e.g. Bindplane) — there is no intermediate gateway, load balancer, or fan-out tier between the source systems and Chronicle.

3.2 Log Source Inventory

Five log sources are currently connected. This is the complete ingestion footprint for this project — any detection rule not mapped to one of these five sources cannot fire in this environment, regardless of whether it appears "active" in the platform’s curated rule library.

Log Source	What it Captures	Ingestion Method
GCP Cloud Audit	Admin Activity, Data Access, System Event and Policy audit logs across GCP projects — IAM policy changes, resource lifecycle events, Compute/Storage/BigQuery/Secret Manager operations, and GKE control-plane (RBAC / cluster role) events.	Native Google-documented export feed (Cloud Logging → Chronicle)
SentinelOne Alerts	Endpoint threat detections (malware, infostealer, virus classifications) from the SentinelOne EDR platform.	Native Google-documented API feed
GitHub	Repository and organization audit events — clones, downloads, PAT (personal access token) usage, collaborator changes, branch protection changes.	Native Google-documented API feed

Log Source	What it Captures	Ingestion Method
Okta	Core identity audit log — authentication, MFA, admin console access, device enrollment, push-notification events.	Native Google-documented API feed
Okta User Context	Supplementary user/session context published alongside core Okta events; enriches entities rather than generating standalone alerts.	Native Google-documented API feed

Note: GKE (Kubernetes) control-plane events — including RBAC/cluster-role changes — are not a separately ingested source. They arrive as part of GCP Cloud Audit logs. An earlier internal analysis mis-bucketed the "Cluster Role Created with Admin Privileges" rule under a standalone "Kubernetes" source; it is correctly a GCP Cloud Audit-sourced detection and is shown as such in Section 4.2.

4. Detection Engineering

4.1 Detection Types

Curated detections: rules shipped and maintained by Google's threat research team, covering a very wide range of integrations. Many ship pre-enabled regardless of whether the underlying source is ingested in a given environment — a curated rule only ever fires if matching UDM data is actually present.

Custom detections: rules authored for this project against the five sources actually ingested (Section 3). These are the rules generating the cases documented below.

4.2 Active Rule Catalog — This Project

The following 29 rules are the ones actually producing cases in this environment, drawn from case export data covering the period 13 Jul – 1 Aug 2026. "Cases" is the number of distinct SOAR cases the rule generated; "Alerts" is the underlying alert volume rolled into those cases; "Avg Risk" is the case-level 0–100 risk score Google SecOps assigns (0 where no risk-scored alerts occurred for that rule in the period).

Source	Rule	Cases	Alerts	Avg Risk
GCP	Gcp Pubsub Createtopic	889	16,412	0.5
GCP	Gcp Setiampolicy Add Binding	807	15,198	3.5
GCP	Gcp Restart Instance	142	2,402	0.0
GCP	Gcp Setiampermissions	95	1,305	0.0
GCP	Gcp Uploadmodel	7	120	0.0
GCP	Gcp Create New Service Account	13	68	0.0
GCP	Gcp Firewall Rule Update	12	46	0.0
GCP	Gcp Deploymodel	2	40	0.0
GCP	Gcp Setcommoninstancemetadata	7	38	0.0

Source	Rule	Cases	Alerts	Avg Risk
GCP	Gcp Storage Bucket Update	3	30	0.0
GCP	Gcp Firewall Rule Created	6	26	0.0
GCP	Gcp Updatefunction	1	20	0.0
GCP	GCP Secret Manager Mass Deletion	3	7	35.0
GCP	Gcp Create New Service Account Key	1	2	0.0
GCP *	Cluster Role Created with Admin Privileges	4	12	35.0
Okta	Okta Auth Via Mfa Non Basic	883	3,689	5.9
Okta	Okta Anomalous Admin Console Access	16	230	0.0
Okta	Okta Access Admin App	27	160	0.0
Okta	Okta Device Enrollment Create	8	112	10.8
Okta	Okta Mismatch Between Source And Response For Verify Push Request	10	74	11.6
Okta	Okta Mfa Factor Activate	3	42	16.2
Okta	Okta User Rejected Multiple Push Notifications	2	18	5.0
Okta	Okta Report Suspicious Activity By Enduser	1	8	0.0
Okta	Okta Detect Client Roaming	1	1	0.0
Okta	Okta New API Token Created	1	1	0.0
GitHub	Github Repository Downloaded by PAT User	7	9	65.0
SentinelOne	SentinelOne Threats: Medium Threat	6	11	70.0
SentinelOne	SentinelOne Threats: Critical Threat	1	2	0.0

Source	Rule	Cases	Alerts	Avg Risk
SentinelOne	SentinelOne Threats: Low Threat	1	1	0.0

Please refer to [this link](#) for the detailed analysis till July 31 2026. The marked rows in the tab “GSecOps Case Detail” in [this report](#) has been identified as the false positive(part of the normal operations)

** GCP Cloud Audit via GKE control-plane events — see the Section 3.2 note.*

4.3 Curated Detection Library (Reference Only)

Beyond the 29 project-specific rules above, the platform ships a much larger curated detection library — well over a thousand named rules spanning dozens of integrations (AWS GuardDuty, Azure Defender for Cloud, CrowdStrike, Google SCC, O365, network/OT vendors such as Nozomi and Claroty, and many malware-family-specific detections). That full catalog is tracked in a separate spreadsheet outside this handbook and is not reproduced or analyzed here. As a general principle: a curated rule appearing "active" in the platform is not evidence that it is currently useful in this environment — only rules mapped to the five ingested sources in Section 3.2 can realistically ever fire.

Please refer to the tab “Secops-MitreMapping” of [this sheet](#) to get the details of the enabled rules(filter for the alerts that has turned ON)

5. SOAR Playbooks

Three playbooks are currently configured. All three run as non-automatic (analyst-confirmed) workflows at Priority 2, scoped to the Default Environment.

5.1 Case Tagging

Field	Value
Trigger	Fires on every incoming case (no filter conditions) — acts as a universal source-labeling step.
Purpose	Labels each case with which log source(s) its underlying events came from, so downstream playbooks and analysts can filter/route by source at a glance.

Logic: a Condition step branches on the event's log type and applies the matching tag:

Event Log Type	Tag Applied
GCP_CLOUDAUDIT	GCP-Entity
OKTA or OKTA_USER_CONTEXT	Okta-Entity
GITHUB	GitHub-Entity

5.2 Okta User Enrichment

Field	Value
Trigger	Alert Name contains "Okta" AND "Admin" — i.e. admin-console-access-flavored Okta alerts.
Purpose	Adds investigative context to admin-console/suspicious-activity Okta cases, filters out confirmed false positives, and fast-tracks genuine cases into the Investigation stage.

Step-by-step flow:

- 1. Add a case comment: "Admin console accessed or Suspicious activity observed."
- 2. Assign the case to the designated on-call analyst.
- 3. Condition: is the underlying event's threat status FALSE_POSITIVE or SUSPICIOUS?
 - → If yes: tag the case "Okta_Investigation".
- 4. Condition: is the alert name a match for Okta AND the alert product Okta?
 - → If yes: move the case to the "Investigation" stage.

5.3 Github Case Assignment

Documentation note: this playbook's stored Description field reads "Enriches Okta-related alerts with user data..." — that text is a leftover copy from the Okta playbook and does not describe what this playbook actually does. The description below reflects the playbook's actual step logic; recommend correcting the stored description in the SOAR console.

Field	Value
Trigger	Alert Name contains "Github" AND "PAT" — i.e. personal-access-token-flavored GitHub alerts.

Field	Value
Purpose	Confirms the alert represents an actual repository download/clone via PAT, applies a tiered SLA based on how the token was used, and escalates further if the actor is a bot account.

Step-by-step flow:

- 1. Condition: does the event's product event type contain "repo.download_zip"?
 - → If yes, proceed; otherwise the playbook takes its default (no-op) branch.
- 2. Move the case to the "Investigation" stage.
- 3. Assign the case to the designated on-call analyst.
- 4. Set a baseline SLA: 3 hours to resolution, 2 hours to critical escalation.
- 5. Condition: does the event's programmatic access type contain "Personal access token"?
 - → If yes, proceed with the branch below; otherwise skip to step 10.
- 6. Tag the case "Github_Investigation".
- 7. Tighten the SLA: 1 hour to resolution, 30 minutes to critical escalation.
- 8. Condition: is the acting user flagged as a bot account?
 - → True and False branches both continue below — both are handled explicitly.
- 9. Re-assign the case (ensuring bot-driven activity still reaches the on-call analyst).
- 10. Add a case comment: "Github repository download observed."
- 11. Move the case to the "Investigation" stage.

5.4 Design Observations

- All three playbooks share the same integration instance and a fixed "assign to" user — there is currently no round-robin or workload-aware assignment logic.

- SLA tiers are hard-coded per playbook rather than driven by the case's risk score, so two cases with very different risk scores from the same rule (e.g. GitHub PAT downloads range from a 65 avg-risk rule to potentially higher/lower per-case scores) currently get the same SLA treatment.
- The Case Tagging playbook's log-type branch list (GCP_CLOUDAUDIT / OKTA / OKTA_USER_CONTEXT / GITHUB) does not yet include a SentinelOne branch — SentinelOne-sourced cases pass through without a source tag applied.

6. Case Management & Triage Workflow

6.1 Case Lifecycle

Cases observed in this environment move through the following stages, driven by the playbooks in Section 5:

Stage	Entered By
(Default / new case)	Automatic on alert-to-case grouping.
Investigation	Okta User Enrichment (on Okta+Admin match) and Github Case Assignment (on repo.download_zip match).

6.2 Tagging Taxonomy

Tag	Applied By	Meaning
GCP-Entity	Case Tagging	Case involves a GCP Cloud Audit-sourced entity.
Okta-Entity	Case Tagging	Case involves an Okta or Okta User Context-sourced entity.
GitHub-Entity	Case Tagging	Case involves a GitHub-sourced entity.
Okta_Investigation	Okta User Enrichment	Underlying event was flagged FALSE_POSITIVE or SUSPICIOUS by threat status — case requires investigation.
Github_Investigation	Github Case Assignment	Confirmed PAT-based repository download — case requires investigation.

6.3 SLA Tiers

Scenario	SLA to Resolution	SLA to Critical
GitHub PAT repo download (baseline)	3 hours	2 hours
GitHub PAT repo download (confirmed PAT usage)	1 hour	30 minutes

7. Best Practices:

7.1 Log Ingestion

- Validate parser coverage before trusting a source. A feed showing "ingesting" doesn't guarantee every field you need is parsed into UDM correctly — spot-check a sample of raw vs. parsed events per source after onboarding, especially for less common event types (e.g., GKE control-plane events inside GCP Cloud Audit, which is easy to mistake for a separate source, as we found).
- Track ingestion volume trends, not just on/off status. A sudden drop in EPS from a source (Okta, GitHub, etc.) is often the first sign of a broken feed credential or API rate-limit issue — alert on volume anomalies per source, not just feed health.
- Document the authoritative source list somewhere versioned, and treat it as the single source of truth for "what can this platform actually detect." You already have this instinct — it's why we corrected the Kubernetes/GCP Cloud Audit misclassification. Any rule or playbook mapped to a non-ingested source should be flagged, not silently left active.
- Avoid unnecessary intermediate log-shipping layers unless you have a specific need (multi-destination fan-out, protocol translation, filtering before egress). Fewer moving parts between source and SIEM means fewer failure points and less to troubleshoot during an incident.

7.2 Detection Engineering (YARA-L Rules)

- Separate curated from custom rules explicitly in your documentation, and periodically audit which curated rules are actually eligible to fire given your ingested sources. An "active" rule count is a vanity metric if 90% of those rules target sources you don't ingest.
- Write rule descriptions that state the specific behavior detected, not just the API method name. "Gcp Setiampolicy Add Binding" tells you the method; it doesn't tell a future analyst whether it also catches binding removals (it likely doesn't, based on the name) — that ambiguity should be resolved in the rule's own documentation, not left for someone to guess later (as we had to).
- Use the numeric risk score as your primary tuning signal, not just the priority label. Priority buckets (Low/Medium/Critical) are coarse; the 0–100 risk score lets you catch a rule that's technically "Low priority" but consistently scoring high risk (or vice versa) — worth a scheduled review, not just point-in-time triage.

- Track alert-to-case ratio per rule. A rule like [Gcp Pubsub Createtopic](#) generating 16,000+ alerts across 889 cases is a strong noise signal — either the rule needs tighter conditions, or it needs to be explicitly marked as low-signal/context-only rather than treated as equivalent to a rule that fires rarely but with high risk (like the GitHub PAT rule at 65 avg risk).
- Version and changelog your custom rules the same way you would code — who changed the logic, when, and why. Detection logic silently drifting is one of the hardest things to debug months later.

7.3 SOAR Playbooks

- Keep playbook descriptions in sync with actual logic, and treat a stale description as a P2 documentation bug, not cosmetic — we already found one (Github Case Assignment's description copy-pasted from the Okta playbook). Add a review step to your playbook change process that forces the description to be re-checked whenever steps change.
- Drive SLAs from the case's risk score where possible, rather than hardcoding them per playbook. Right now your GitHub playbook applies the same SLA tier regardless of whether the underlying case scored a risk of 20 or 90 — a risk-score-driven SLA (e.g., tiered thresholds) scales better as you add more rules under the same playbook.
- Give every ingested source an explicit tagging branch. Your Case Tagging playbook currently has no SentinelOne branch, so those cases pass through untagged — any log source you ingest should be reachable by every playbook that does source-based routing, or analysts lose the ability to filter by it.
- Avoid single-user hardcoded assignment. All three of your playbooks assign to one fixed user — this doesn't scale past one analyst and creates a single point of failure during PTO/on-call handoff. Round-robin or on-call-schedule-aware assignment (via a dynamic list/lookup) is worth planning even if you don't implement it immediately.
- Design condition branches to have an explicit default, not just a happy path. Your existing playbooks generally do this well (explicit True/False branches on bot detection, for example) — keep that pattern as you add more playbooks.

7.4 Case Management & Triage

- Standardize your tagging taxonomy before it grows organically. You already have a clean split (*-Entity tags for source, *_Investigation tags for confirmed-worthy cases) — document that

convention explicitly so future playbooks follow the same pattern instead of inventing ad hoc tag names.

- Review SLA tiers against real MTTR data periodically. An SLA that's aspirational rather than grounded in actual analyst throughput just generates chronic SLA breaches, which erodes the metric's usefulness.
- Use case stage transitions consistently — if "Investigation" is meant to mean "a human needs to look at this," don't let playbooks move cases there for purely informational reasons; it dilutes the signal for anyone using stage as a triage filter.

7.5 General Platform Hygiene

- Re-run a "which of these rules can actually fire" audit whenever you onboard a new log source — new sources unlock previously-dormant curated rules, and you'll want to know that's happened rather than discover it via a surprise alert spike.
- Keep a single changelog for the whole environment (ingestion changes, rule changes, playbook changes) rather than three separate untracked histories — most real incidents in a SOAR/SIEM environment trace back to "what changed recently," and a fragmented history makes that harder to answer under pressure.

8. Appendices

Appendix A — Curated Detection Rules Catalog

The full list of curated detection rules currently marked active in the platform (spanning integrations well beyond the five sources ingested in this project) is maintained separately as an Excel workbook and will be attached alongside this handbook. It is referenced here for completeness but intentionally not reproduced or analyzed in this document — see Section 4.3.

Appendix B — Glossary

Term	Definition
UDM	Unified Data Model — Chronicle's normalized schema that all ingested logs are parsed into.
YARA-L	The rule language used to author custom Chronicle/Google SecOps detections.
Curated detection	A pre-built rule authored and maintained by Google, shipped as part of the platform's content library.
Case	A SOAR-level grouping of one or more related alerts, carrying a priority, risk score, stage, and assignee.
Alert	A single detection-rule match against one or more UDM events.
Playbook	An automated SOAR workflow that reacts to case/alert triggers and executes a sequence of actions.
Feed	Google SecOps' native, documented mechanism for pulling or receiving logs from a given source product.
Risk Score	A 0–100 numeric score Google SecOps assigns to alerts/cases, offering finer-grained prioritization than a flat severity label.